

We'll treat each line of code as one step except for the `for` loop. This line executes once for each item in `books`, and because the size of `books` is our n , we say that this executes n steps. Not only that, but it executes all the steps inside the loop n times. Because there are two steps inside the loop, the total is $2 \times n$ steps. We could describe our steps like this:

```
def readingList(books):
    print('Here are the books I will read:') # 1 step
    numberOfBooks = 0                        # 1 step
    for book in books:                      # n * 2 steps
        print(book)                         # (already counted)
        numberOfBooks += 1                  # (already counted)
    print(numberOfBooks, 'books total.')    # 1 step
```

Now when we compute the total number of steps, we get $1 + 1 + (n \times 2) + 1$. We can rewrite this expression more simply as $2n + 3$.

Big O doesn't intend to describe specifics; it's a general indicator. Therefore, we drop the lower orders from our count. The orders in $2n + 3$ are linear ($2n$) and constant (3). If we keep only the largest of these orders, we're left with $2n$.

Next, we drop the coefficients from the order. In $2n$, the coefficient is 2. After dropping it, we're left with n . This gives us the final big O of the `readingList()` function: $O(n)$, or linear time complexity.

This order should make sense if you think about it. There are several steps in our function, but in general, if the `books` list increases tenfold in size, the runtime increases about tenfold as well. Increasing `books` from 10 books to 100 books moves the algorithm from $1 + 1 + (2 \times 10) + 1$, or 23 steps, to $1 + 1 + (2 \times 100) + 1$, or 203 steps. The number 203 is roughly 10 times 23, so the runtime increases proportionally with the increase to n .

Why Lower Orders and Coefficients Don't Matter

We drop the lower orders from our step count because they become less significant as n grows in size. If we increased the `books` list in the previous `readingList()` function from 10 to 10,000,000,000 (10 billion), the number of steps would increase from 23 to 20,000,000,003. With a large enough n , those extra three steps matter very little.